Ramu Roy

Embedded Systems Engineer

ramuroy | Ramu Roy | royramu694429@gmail.com | +91 94936 52315

Hyderabad, India | ramuroy.github.io

Summary

Embedded systems engineer (B.Tech ECE, 2026, **RGUKT Srikakulam**, CGPA 8.3/10) who owns the whole stack — PCB design through firmware to custom embedded Linux. At Elipse I design the eOS fleet's control boards in **KiCad**, schematic through layout to the fabrication package — a 24 V room controller fabricated and in service, a 48 V sixteen-channel floor controller, and a SELV wall keypad — and contribute to **eOS** itself, a Yocto/OpenEmbedded Linux distribution for Raspberry Pi 5 with A/B RAUC OTA, an MQTT service bus, a Rust per-room sensor framework, and an on-device Rust voice stack (wake-word, Whisper STT, Piper TTS). Outside work I have written a KiCad-native PCB autorouter and a personal OS with a hand-written Rust init and shell. Previously, during a six-month engagement at Radiogeet, I built the ESP32-S3 firmware for an industrial UWB Anti-Collision System now deployed at **Tata Steel Colors (formerly Tata BlueScope Steel)**.

Education

2022 – 2026 Bachelor's in Electronics and Communication Engineering at **RGUKT Srikakulam**, Andhra Pradesh (CGPA: 8.3/10)

2020 – 2022 Pre-University Course at **RGUKT Srikakulam**, Andhra Pradesh (GPA: 9.63/10)

Work Experience

Embedded Systems Engineer, Elipse May 2026 – Present

– **eOS — Custom Linux Distribution on Raspberry Pi 5 (16 GB RAM)**
Contributing to a Yocto Project / OpenEmbedded-based custom embedded Linux operating system targeting Raspberry Pi 5, featuring an A/B RAUC OTA-update rootfs, MQTT-based service bus, SQLite persistence, and a Qt6/QML local UI.

– Author and extend Yocto recipes (.bb / .bbappend) across the meta-eos layer for new system services, ML model assets, configuration, and ACL policies; manage BitBake PR bumps, AUTOREV pinning, and IPK packaging for incremental on-device deployment.

– Work across the deployed Linux subsystem stack: systemd unit design and service lifecycle, D-Bus interface authoring (org.eos.Config1, RoomCommands1, RoomAggregates1), a Mosquitto MQTT broker hardened with TLS and ACLs, and SQLite schema migrations with busy-timeout concurrency tuning for multi-writer workloads.

– Design and build a generic Rust RoomAggregator framework for per-room sensor fusion (thermal, mmWave radar, air quality, ambient light, PIR motion) with calibration via D-Bus and SQLite-backed persistence.

– Build an on-device voice subsystem: a transfer-learning wake-word detector (Google speech_embedding backbone + PyTorch-trained head, ONNX export, tract pure-Rust runtime), multi-microphone best-source fusion across ESP32 satellites, Whisper STT, and Piper TTS with async-Rust barge-in interruption.

– **ESP32 Firmware (ESP-IDF)**

– Develop firmware for ESP32 sensor-satellite devices using ESP-IDF v5.2, including BLE-based provisioning with on-chip EC P-256 keypair generation, X.509 CSR exchange with the hub CA, and full NVS lifecycle management (factory reset, identity preservation across OTA flashes).

– Implement pin-keyed sensor MQTT topics for disambiguating same-role sensors across GPIOs, an SNTP-synced audio-chunk frame format for streaming microphone audio over the network, and fixes for sensor-

task scheduler bugs (null-queue panic loops, digital-event drain races).

– **Build System & Tooling**
– Extend kas build orchestration and the in-house eos-build CLI with host-specific overlay support, on-demand firmware builds, and PR-bump automation across the Yocto layer.
– Own end-to-end deployment flow: cross-compilation, full WIC image builds, bmaptool SD-card flashing, and RAUC A/B verification.

– **Hardware / PCB Design**
– Design the fleet's control boards in **KiCad** end to end — schematic through layout, DRC and the fabrication package — across three boards: a 24 V per-room controller, a 48 V sixteen-channel floor controller, and a SELV wall keypad.
– **Room Controller:** v1 fabricated and put into service; v2 re-spun at 125×100 mm (**−57 % board area**) with eight dimmable 24 V PWM channels, two addressable RGB outputs, and a five-stage input protection chain (fuse → 60 V eFuse → TVS → tap fuse → 5 V eFuse) replacing v1's unprotected rail. Ran the PWM expander at 5 V to drive the FET gates directly, deleting four gate-driver ICs. v2 is live in three rooms.
– **Zone Controller:** 48 V, sixteen 12-bit PWM channels at 1.5 A each (6 A per four-channel group, 24 A / 1,152 W board), on four layers of an impedance-controlled stackup with 100 Ω differential Ethernet pairs. Made the ESP32 the RMII clock master, deleting a gated clock buffer and four support parts. Routing closed at **0 unconnected**; DC review on the final copper gives 19.9 mV worst driver distribution against a 50 mV bound; fabrication package cut.
– **Switchboard:** SELV wall control surface, 145×70 mm two-layer, ESP32-S3, 30 addressable indicators, CAN uplink — fabricated, in bring-up.
– Bench bring-up and rework across all three boards: power-path debug, fault isolation, and repair.

Embedded Engineer, Radiogeet Sep 2025 – Mar 2026

– **Project: Industrial Anti-Collision System**
Developed an industrial Anti-Collision System for crane operations, deployed at **Tata Steel Colors** (formerly Tata BlueScope Steel). The system performs real-time UWB-based proximity detection, executes zone-based safety logic, and triggers industrial outputs to prevent hazardous crane movements.
– Designed and implemented firmware on ESP32-S3 leveraging its dual-core architecture.
– Allocated one core for time-critical UWB distance measurement and the second core for zone calculation, system logic, and embedded web interface.
– Implemented ESP-NOW for low-latency peer-to-peer wireless communication between system nodes.
– Established MODBUS RTU over RS485 communication with Masibus DO cards to control an 8-channel industrial relay system.

– **Additional Projects and Contributions**
– Interfaced AHT10 sensors and ADS1115 external ADCs with STM32 microcontrollers; configured timers, internal ADC and DAC in STM32CubeIDE.
– Implemented two- and four-wire RS485 for industrial data exchange, LoRa long-range links, and Masibus DI/DO/AI/AO industrial cards.
– Wrote firmware across blocking and non-blocking delays, polling, and interrupt-driven designs.

Research and Development Engineer Intern, Ampnics Mar 2025 – Sep 2025 (Hybrid)

– Designed and reviewed PCB schematics and layouts for open-source hardware projects.
– Supported rapid prototyping through circuit testing, debugging, and iterative design improvements.

Projects

pcbrouter — KiCad-native PCB autorouter and routing verifier (Rust) (2026)
A from-scratch autorouter built on exact integer geometry: board coordinates parse to integer nanometres and clearance predicates run on i128 and 256-bit rationals, so no clearance decision depends on floating point. Proves its model against KiCad before routing — loader dump field-identical to pcbnew across 1,212 footprints and 4,514 pads — and checks its own output afterwards with KiCad's DRC. Negotiated-congestion routing over a tile graph with exact capacities; benchmarked on a commit-pinned corpus of published designs (HackRF One, Bus Pirate 5, Olimex ESP32-PoE). Apache-2.0.

Ember — personal OS with a hand-written Rust init and shell (2026)
An existing Linux kernel and a reproducible Yocto userland with systemd removed and replaced by two programs written from scratch: `spark`, a PID 1 and service manager in a single epoll loop (dependency graph, readiness-driven startup, restart backoff, cgroup v2 per service), and `hearth`, the login shell. Boots an HP Victus over UEFI with signed A/B RAUC updates and unaided rollback. 341 workspace tests; five audited `unsafe` blocks, with all three libraries `forbid(unsafe_code)`.

Skills

Languages	C, Embedded C/C++, Rust (async / Tokio), Python, MATLAB
PCB & Hardware	KiCad — schematic capture, layout, ERC/DRC, impedance-controlled stack-ups, 100 Ω differential-pair routing, eFuse protection design, power-supply design, gerber/CAM release, BOM & sourcing, bring-up, rework and fault-finding; Raspberry Pi 5, ESP32 / ESP32-S3, STM32
Embedded Linux & OS	Yocto Project, BitBake, OpenEmbedded, kas, meta-layers, .bb/.bbappend recipes, IPK packaging, RAUC A/B OTA, custom distro build, WIC images, GRUB-EFI/UEFI boot, cross-compilation, BSP, device tree; systemd unit design, D-Bus interface authoring, journald, SQLite (schema, migrations, concurrency tuning), Mosquitto MQTT (TLS, ACL)
Firmware & RTOS	ESP-IDF v5.x (ESP32 / ESP32-S3), FreeRTOS dual-core task allocation, STM32 HAL/LL via STM32CubeIDE (timers, ADC, DAC, GPIO, UART), bare-metal, interrupt-driven & polling designs, BLE provisioning (NimBLE), NVS lifecycle, OTA
On-Device ML & Voice	PyTorch, ONNX, tract (Rust ONNX runtime), transfer learning, keyword spotting (KWS), VAD, mel-spectrogram features, Whisper STT, Piper TTS, multi-mic best-source fusion, barge-in
Protocols	UART, SPI, I^2C, CAN, RS485 (MODBUS RTU), MQTT, D-Bus, BLE, Wi-Fi, ESP-NOW, LoRa, UWB
Sensors & Tooling	HLK-C4001 mmWave radar, PIR motion, PM/VOC/CO_2 air quality, SHT40/AHT10, TSL2591, ADS1115, INA226/INA238, PCA9685, UWB modules, Masibus DI/DO/AI/AO cards, relays; Qt6/QML, Git/GitHub, pcbnew scripting

Certifications & Languages

2025	**NPTEL: Embedded Sensing, Actuation and Interfacing Systems** — certificate (Score: 85%)
2025	**NPTEL: Electronic Systems Design — Circuits & PCB Design** — certificate (Score: 88%)
Languages	English (Proficient), Telugu (Native), Hindi (Native), Japanese (Professional working)